Introduction to Quantum Computing

Frédéric Laprade

jean-frederic.laprade@usherbrooke.ca

AlgoLab, Institut quantique, Université de Sherbrooke, Canada

QEst Hack 2026

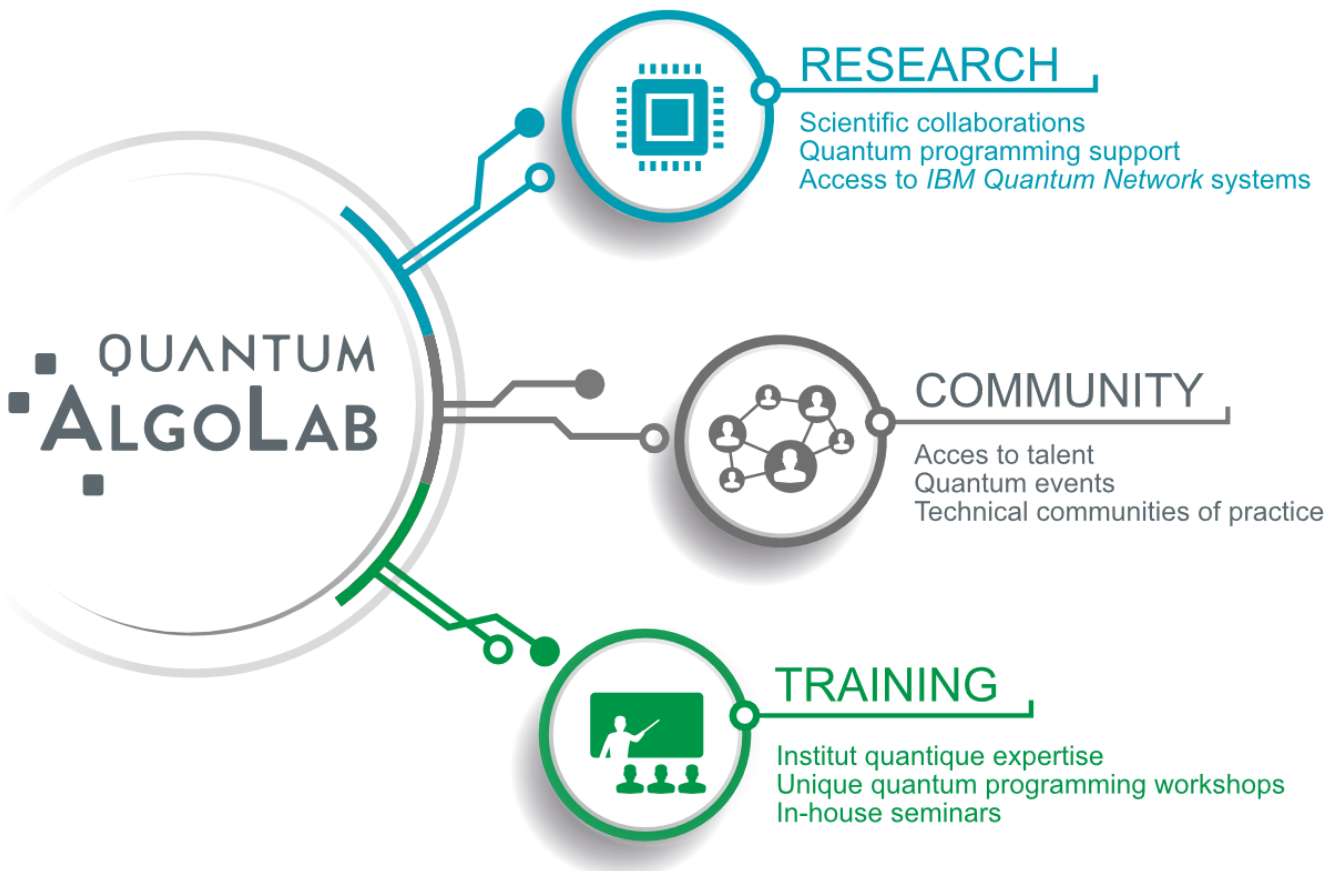
Table of contents

- Quantum AlgoLab
- Overview
- Qubits, gates and quantum circuits
- Measurement, observables and Hamiltonian
- Quantum algorithms
- Quantum combinatorial optimization
- QAOA

Quantum AlgoLab

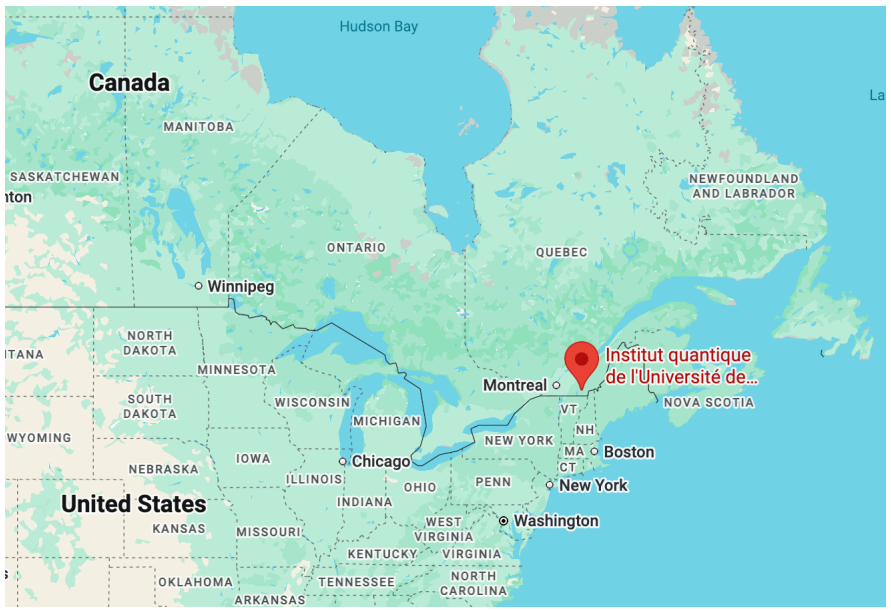
QEst Hack 2026

What we do



QEst Hack 2026

Institut quantique



- 40+ research groups
- 250+ students / postdocs
- 35+ research staff

INSTITUT QUANTIQUE UdS

Overview

Objectives

- Learn the basics of quantum computing in a friendly environment.
- Connect students and businesses
- Get insights from seasoned practitioners
- Position your business with respect to the emergence of quantum technologies
- Express a business problem in a form suitable for quantum algorithms

Prerequisites

- Basic linear algebra (vectors, matrices and linear operations)
- Python coding
- Coffee ☕

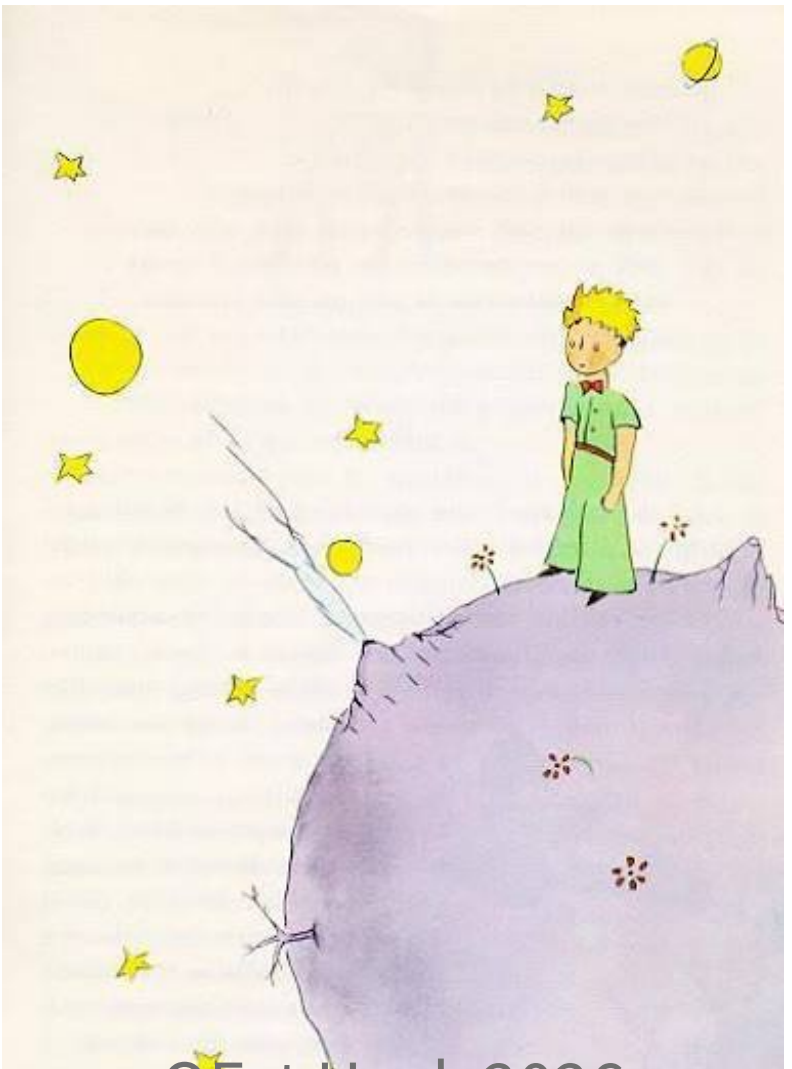
Lessons from last year (1)

One bite at a time!

(Image credit: Thomas Merkt / 500px via Getty Images)

Lessons from last year (2)

Discuss with teammates and mentors!



Lessons from last year (3)

Research can become messy!

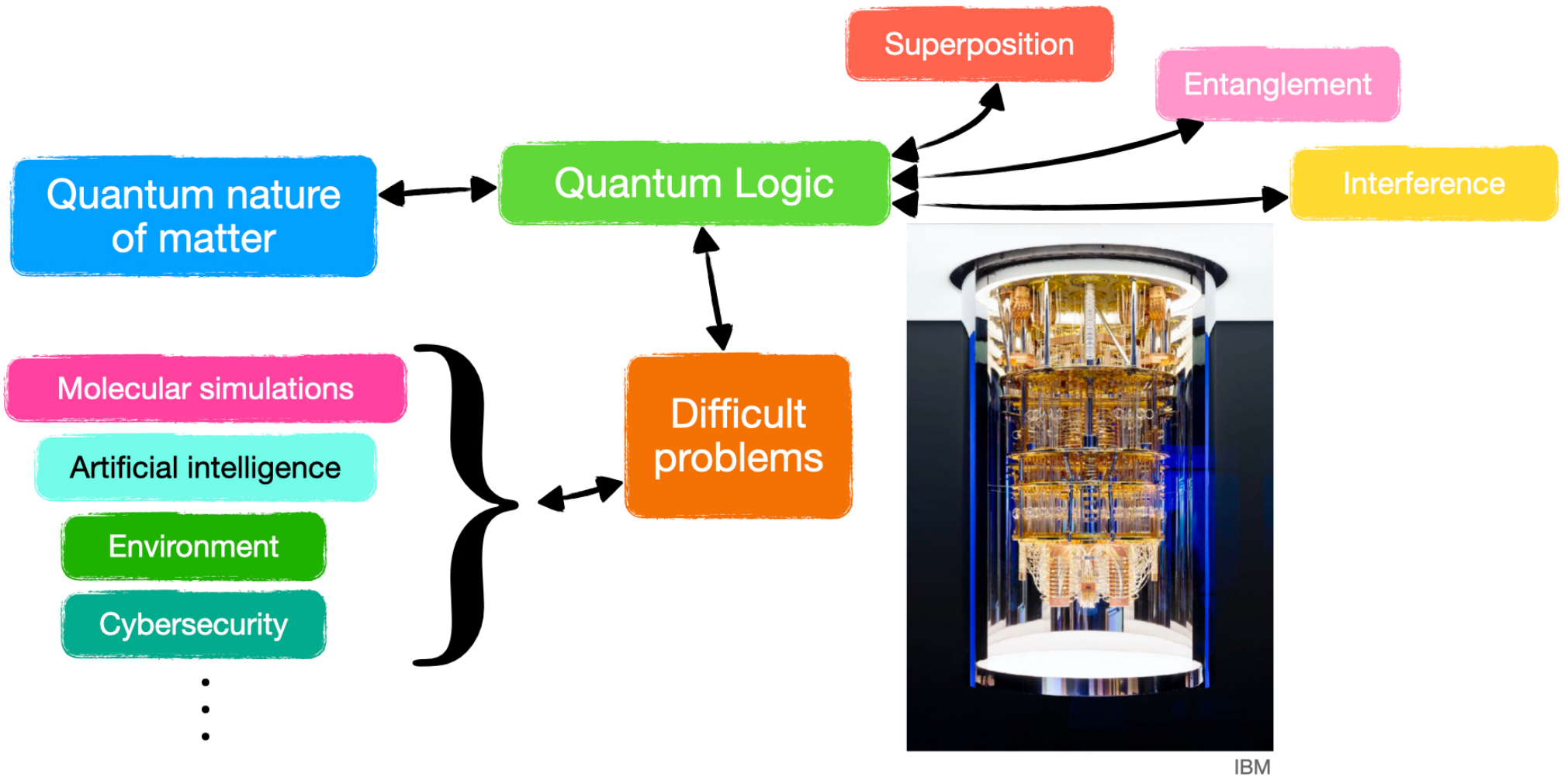
Lessons from last year (3)

Have fun!

Qubits, gates and quantum circuits

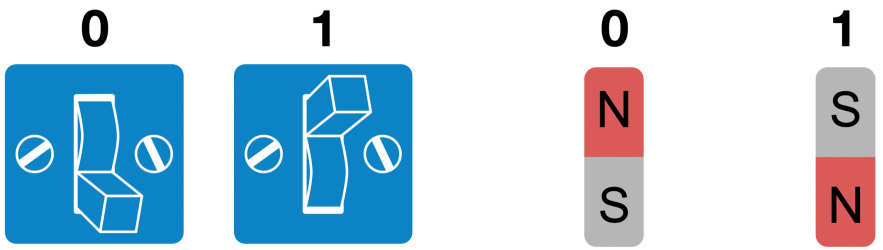
QEst Hack 2026

Quantum logic

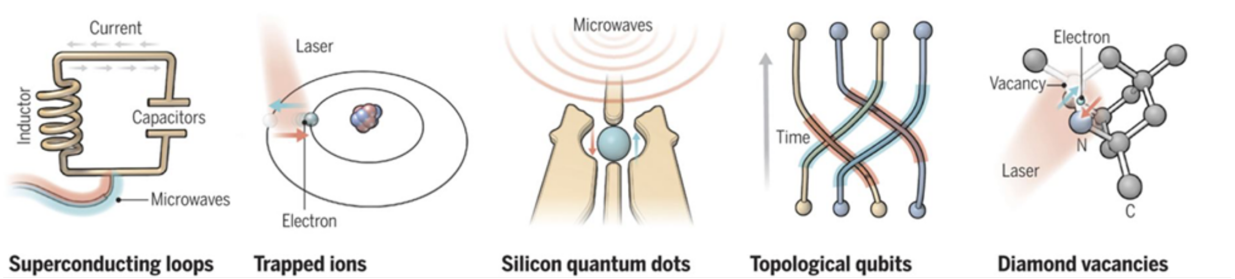


Information

- Classical information is encoded with bits



- Quantum information is encoded with *quantum bits*: qubits



GRAPHIC BY C. BICKLE/SCIENCE DATA BY GABRIEL POPKIN

QEst Hack 2026

Qubit

A qubit is a 2-level quantum system. We label these levels $|0\rangle$ and $|1\rangle$.

A qubit can be in the state $|0\rangle$, $|1\rangle$ or in a superposition of these 2 states.

```python
from qiskit import QuantumCircuit

qc = QuantumCircuit(1)
qc.draw('mpl')
```

State vector - 1 qubit

The state of a qubit is described by a normalized vector

$$|\psi\rangle = \begin{bmatrix} \alpha_0 \\ \alpha_1 \end{bmatrix}$$

with α_0 and α_1 being complex numbers such that $|\alpha_0|^2 + |\alpha_1|^2 = 1$.

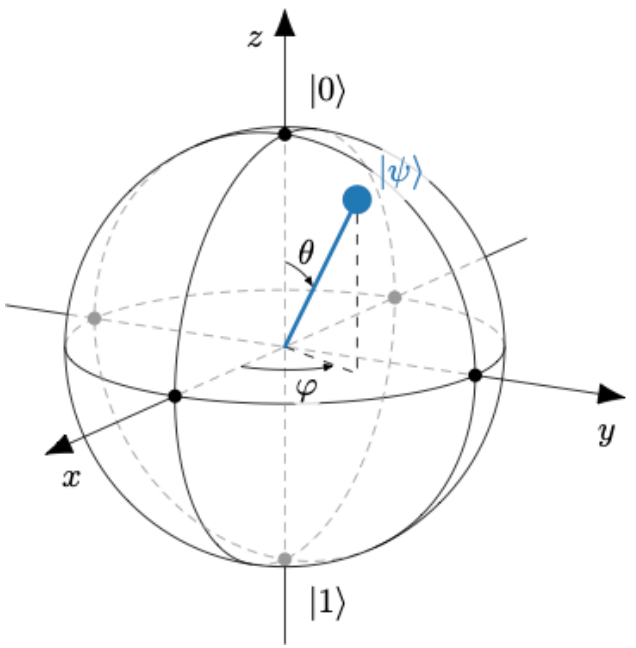
By taking $|0\rangle = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$ and $|1\rangle = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$, we have

$$|\psi\rangle = \alpha_0|0\rangle + \alpha_1|1\rangle.$$

Bloch Sphere

The constraint $|\alpha_0|^2 + |\alpha_1|^2 = 1$ corresponds to the surface of a sphere in parameters space.

The **Bloch sphere** is a sphere of radius 1 used to represent the state of a qubit.



Dual state

For a state $|\psi\rangle = \begin{bmatrix} \alpha_0 \\ \alpha_1 \end{bmatrix}$, we define $\langle\psi| = [\alpha_0^* \quad \alpha_1^*]$.

We can write the normalization constraint as

$$\begin{aligned} \langle\psi|\psi\rangle &= [\alpha_0^* \quad \alpha_1^*] \begin{bmatrix} \alpha_0 \\ \alpha_1 \end{bmatrix} \\ &= |\alpha_0|^2 + |\alpha_1|^2 \\ &= 1. \end{aligned}$$

State vector - 2 qubits

A two-qubit system is described by a 4-dimensional vector:

$$|\psi\rangle = \begin{bmatrix} \alpha_0 \\ \alpha_1 \\ \alpha_2 \\ \alpha_3 \end{bmatrix} = \alpha_0|00\rangle + \alpha_1|01\rangle + \alpha_2|10\rangle + \alpha_3|11\rangle,$$

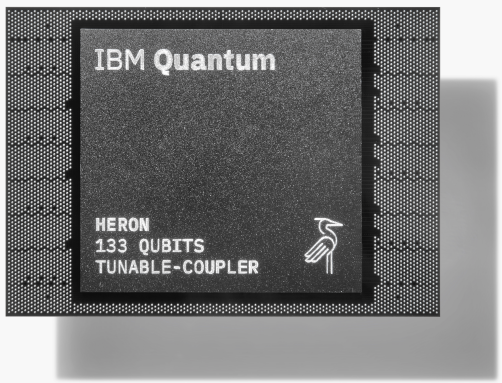
with $\alpha_i \in \mathbb{C}$ and $\sum_{i=0}^{3} |\alpha_i|^2 = 1$.

Equivalently, $|\psi\rangle = \alpha_0|0\rangle + \alpha_1|1\rangle + \alpha_2|2\rangle + \alpha_3|3\rangle$.

State vector - N qubits

For N qubits,

$$|\psi\rangle = \sum_{i=0}^{2^N-1} \alpha_i |i\rangle$$



with

$$\sum_{i=0}^{2^N-1} |\alpha_i|^2 = 1$$

State-space size:

$$2^{133} = 10889035741470030830827987437816582766592$$
$$\sim 10^{31}\text{GB}$$

Quantum gates

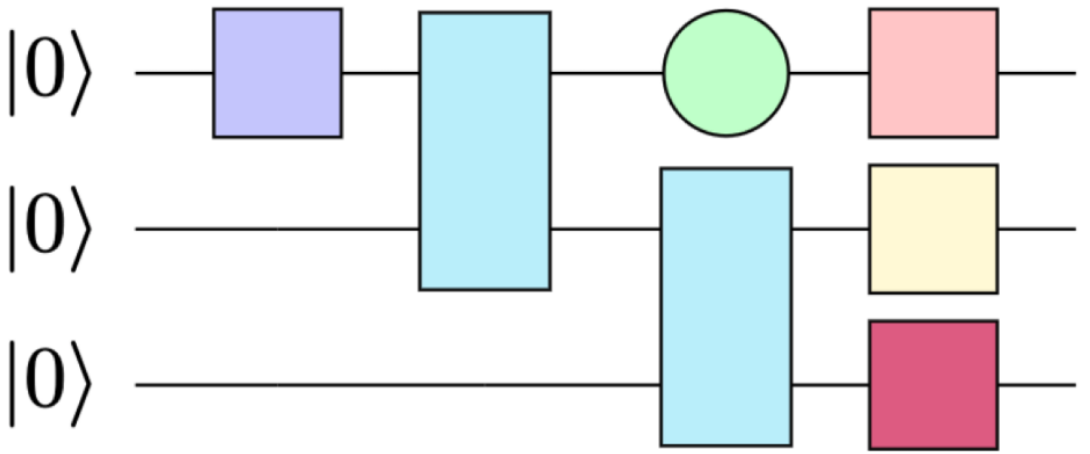
By convention, qubits are initialized in the $|0\rangle$ state.

We modify the state of a qubit (i.e., we perform calculations!) by applying **quantum gates**.

Mathematically, a **quantum gate** is a **unitary matrix**, i.e. it preserves the unitary constraint of quantum states.

Quantum circuits

A **quantum circuit** is a sequence of quantum gates applied to a system of qubits.



Abstraction of a quantum circuit with 3 qubits

- One line per qubit
- Read from left to right
- A gate can apply to more than one qubit

X Gate

$$X = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$$

$$X|0\rangle \rightarrow |1\rangle$$

$$X|1\rangle \rightarrow |0\rangle$$

$$\begin{aligned} X(\alpha|0\rangle + \beta|1\rangle) \\ &\rightarrow \alpha\, X|0\rangle + \beta\, X|1\rangle \\ &\rightarrow \beta|0\rangle + \alpha|1\rangle \end{aligned}$$

XGate

```
1  qc = QuantumCircuit(1)
2
3  qc.x(0)
4
5  qc.draw('mpl')
```

Z Gate

$$Z = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$$

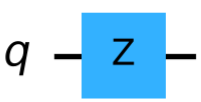
$$Z|0\rangle \rightarrow |0\rangle$$

$$Z|1\rangle \rightarrow -|1\rangle$$

$$\begin{aligned} Z(\alpha|0\rangle + \beta|1\rangle) \\ &\rightarrow \alpha\, Z|0\rangle + \beta\, Z|1\rangle \\ &\rightarrow \alpha|0\rangle - \beta|1\rangle \end{aligned}$$

ZGate

```
1  qc = QuantumCircuit(1)
2
3  qc.z(0)
4
5  qc.draw('mpl')
```



QEst Hack 2026

H Gate

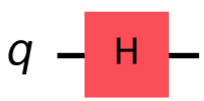
$$H = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$$

HGate

```
1  qc = QuantumCircuit(1)
2
3  qc.h(0)
4
5  qc.draw('mpl')
```

$$H|0\rangle \rightarrow 1/\sqrt{2}\,(|0\rangle + |1\rangle)$$

$$H|1\rangle \rightarrow 1/\sqrt{2}\,(|0\rangle - |1\rangle)$$

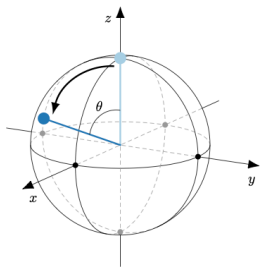


$$H(\alpha|0\rangle + \beta|1\rangle)$$
$$\rightarrow \alpha\, H|0\rangle + \beta\, H|1\rangle$$
$$\rightarrow \Big((\alpha+\beta)/\sqrt{2}|0\rangle + (\alpha-\beta)/\sqrt{2}|1\rangle\Big)$$

Arbitrary rotation gates

RXGate

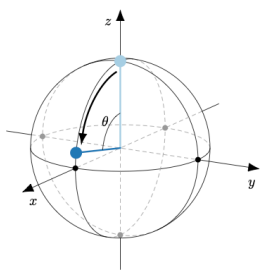
$$RX(\theta) = \begin{bmatrix} \cos\left(\frac{\theta}{2}\right) & -i\sin\left(\frac{\theta}{2}\right) \\ -i\sin\left(\frac{\theta}{2}\right) & \cos\left(\frac{\theta}{2}\right) \end{bmatrix}$$

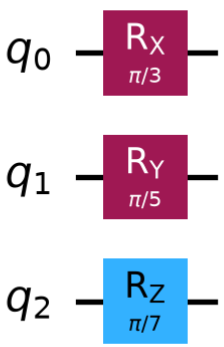


```python
qc = QuantumCircuit(3)

qc.rx(np.pi/3, 0)
qc.ry(np.pi/5, 1)
qc.rz(np.pi/7, 2)

qc.draw('mpl')
```

RYGate

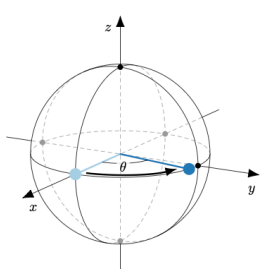
$$RY(\theta) = \begin{bmatrix} \cos\left(\frac{\theta}{2}\right) & -\sin\left(\frac{\theta}{2}\right) \\ \sin\left(\frac{\theta}{2}\right) & \cos\left(\frac{\theta}{2}\right) \end{bmatrix}$$





RZGate

$$RZ(\theta) = \begin{bmatrix} e^{-i\frac{\theta}{2}} & 0 \\ 0 & e^{i\frac{\theta}{2}} \end{bmatrix}$$



Entanglement

Two qubits are entangled when the state of one qubit cannot be described independently of the other.

Entanglement

Two qubits are entangled when the state of one qubit cannot be described independently of the other.

CX Gate

One qubit acts as **control**, the other as **target**: CX_{ct}.

- If the control qubit is in the $|1\rangle$ state, an X gate is applied to the target qubit.
- If the control qubit is in the $|0\rangle$ state, no operation is applied to the target qubit.

$$CX_{01}|00\rangle \rightarrow |00\rangle$$
$$CX_{10}|00\rangle \rightarrow |00\rangle$$

CX Gate

One qubit acts as **control**, the other as **target**: CX_{ct}.

- If the control qubit is in the $|1\rangle$ state, an X gate is applied to the target qubit.
- If the control qubit is in the $|0\rangle$ state, no operation is applied to the target qubit.

$CX_{01}|00\rangle \to |00\rangle$
$CX_{10}|00\rangle \to |00\rangle$

$CX_{01}|01\rangle \to |11\rangle$
$CX_{10}|01\rangle \to |01\rangle$

CX Gate

One qubit acts as **control**, the other as **target**: CX_{ct}.

- If the control qubit is in the $|1\rangle$ state, an X gate is applied to the target qubit.
- If the control qubit is in the $|0\rangle$ state, no operation is applied to the target qubit.

$$CX_{01}|00\rangle \to |00\rangle \quad CX_{01}|01\rangle \to |11\rangle \quad CX_{01}|10\rangle \to |10\rangle$$
$$CX_{10}|00\rangle \to |00\rangle \quad CX_{10}|01\rangle \to |01\rangle \quad CX_{10}|10\rangle \to |11\rangle$$

CX Gate

One qubit acts as **control**, the other as **target**: CX_{ct}.

- If the control qubit is in the $|1\rangle$ state, an X gate is applied to the target qubit.
- If the control qubit is in the $|0\rangle$ state, no operation is applied to the target qubit.

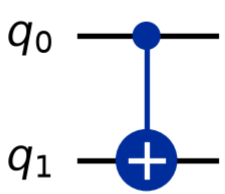
$$CX_{01}|00\rangle \to |00\rangle \quad CX_{01}|01\rangle \to |11\rangle \quad CX_{01}|10\rangle \to |10\rangle \quad CX_{01}|11\rangle \to |01\rangle$$
$$CX_{10}|00\rangle \to |00\rangle \quad CX_{10}|01\rangle \to |01\rangle \quad CX_{10}|10\rangle \to |11\rangle \quad CX_{10}|11\rangle \to |10\rangle$$

CX Gate

One qubit acts as **control**, the other as **target**: CX_{ct}.

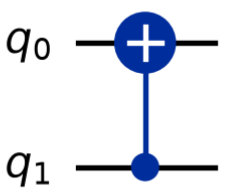
CX_{01}

```
1  qc = QuantumCircuit(2)
2  qc.cx(0, 1)
3
4  qc.draw('mpl')
```



CX_{10}

```
1  qc = QuantumCircuit(2)
2  qc.cx(1, 0)
3
4  qc.draw('mpl')
```



Summary

- A qubit is a **unit of quantum information**.
- The state of a qubit is described by a **two-dimensional unit vector** with complex coefficients.
- The size of the state space **grows exponentially** with the number of qubits.
- The operations that can be applied to qubit states are **unitary matrices called quantum gates**.
- A **quantum circuit** is a series of quantum gates that achieve a computation.

Measurement, observables and Hamiltonian

QEst Hack 2026

Measurement - Born rule

The measurement of a quantum state is **probabilistic**. For a state

$$|\psi\rangle = \sum_{i=0}^{2^N-1} \alpha_i |i\rangle,$$

the probability of measuring the state $|i\rangle$ is $\mathcal{P}_i = |\alpha_i|^2$.

Let $|\psi\rangle = \frac{2}{3}|00\rangle - \frac{1}{3}|01\rangle - \frac{2}{3}|10\rangle$.

If we measure qubits q_0 and q_1, we can obtain the following results

q_0	q_1	**Probability**
0	0	$4/9$
1	0	$1/9$
0	1	$4/9$
1	1	0

Quantum parallelism

Corollary:

*A quantum computer **is NOT** a super-computer with exponentially many cores: the outcome of a computation is a single basis state!*

Measurement - state collapse

- A measurement changes the state of a qubit.
- We prepare the state

$$|\psi\rangle = \sum_{i=0}^{2^N-1} \alpha_i |i\rangle,$$

  and we measure it. The measurement outcome is a value $k \in \{0, 1, \dots, 2^N - 1\}$.

- After the measurement, we have $|\psi\rangle = |k\rangle$.

Measurement - sampling

Since the measurement result is probabilistic, we prepare and measure a quantum state multiple times.

The result is a dictionary of pairs (k_i, v_i) where

- k_i is a measured state $\left(k_i \in \{0, 1, \dots, 2^N - 1\}\right)$,
- v_i is the number of times this state was measured.

For a budget of Γ shots, an estimator for the probability to compute one basis state k_i is :

$$|\alpha_i|^2 \sim \bar{\mathcal{P}}_i = \frac{v_i}{\Gamma}.$$

Measurement - example

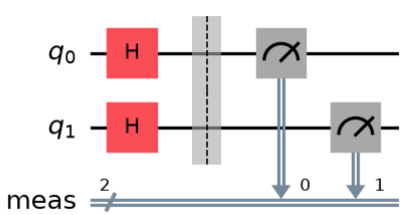
If we prepare and measure the state of equal superposition over 2 qubits, we should expect to obtain

$$\bar{\mathcal{P}}_i = \frac{v_i}{\Gamma} = \frac{1}{4}$$

as $\Gamma \to \infty$.

```python
from qiskit_aer import AerSimulator
from qiskit_ibm_runtime.executor_sample

qc = QuantumCircuit(2)
qc.h([0, 1])
qc.measure_all()

simulator = AerSimulator()
sampler = Sampler(simulator)
result = sampler.run([qc], shots=1000).

print(result[0].data.meas.get_counts())
qc.draw('mpl', scale=0.6)
```

```
{'01': 263, '00': 273, '11': 232, '10':
232}
```



Observable

- In quantum mechanics, an **observable** is a quantity that can be measured, for example the energy of a system.
- Observables are **Hermitian matrices**, i.e. matrices A such that $A_{ij} = A^*_{ji}$. Hermitian matrices have real eigenvalues.
- Given a quantum state $|\psi\rangle$ prepared by an algorithm, the expectation value of the observable $\mathcal{O}$ is

$$\langle \mathcal{O} \rangle_\psi = \langle \psi | \mathcal{O} | \psi \rangle$$

Observable

- In quantum mechanics, an **observable** is a quantity that can be measured, for example the energy of a system.
- Observables are **Hermitian matrices**, i.e. matrices A such that $A_{ij} = A^*_{ji}$. Hermitian matrices have real eigenvalues.
- Given a quantum state $|\psi\rangle$ prepared by an algorithm, the expectation value of the observable $\mathcal{O}$ is

$$\begin{aligned}\langle \mathcal{O} \rangle_\psi &= \langle \psi | \mathcal{O} | \psi \rangle \\ &= \begin{bmatrix} \alpha^* & \beta^* \end{bmatrix} \begin{bmatrix} \mathcal{O}_{00} & \mathcal{O}_{01} \\ \mathcal{O}_{10} & \mathcal{O}_{00} \end{bmatrix} \begin{bmatrix} \alpha \\ \beta \end{bmatrix}\end{aligned}$$

Observables… *Why oh why?*

- We want to find the solution to a binary optimization problem.
- Classically, the value of a solution is measured with a cost function.
- Quantumly, the cost function will be expressed as an observable.
- We'll want to compute the expectation value of the observable. The lower the expectation value, "the closer" the prepared state is to the desired solution: the **ground state**!

Hamiltonian operator

- The operator representing the energy of a quantum system is called the **Hamiltonian**.
- The Hamiltonian must be expressed in a basis that is convenient for a quantum computer: the Pauli basis!
- For the hackathon, we need only two out of the four Pauli matrices:

$$I = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \qquad Z = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$$

Expectation value - 1 qubit

I and Z are diagonal, their eigenvalues are simply the entries on the diagonal. The expectation values are

State	$\langle I \rangle$	$\langle Z \rangle$
$\|0\rangle$	1	1
$\|1\rangle$	1	-1

For an arbitrary state $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$,

$$\begin{aligned}\langle Z \rangle_\psi &= |\alpha|^2 \langle Z \rangle_{|0\rangle} + |\beta|^2 \langle Z \rangle_{|1\rangle} \\ &= |\alpha|^2 - |\beta|^2.\end{aligned}$$

Expectation value - 2 qubits

- For two qubits, we can measure the expectation values of the operators II, IZ, ZI, and ZZ.
- To do this, we multiply the expectation values of the single-qubit operators.

For example,

$$\langle ZI \rangle_{|10\rangle} = \langle Z \rangle_{|1\rangle} \langle I \rangle_{|0\rangle} = (-1)(1) = -1$$

$$\langle ZZ \rangle_{|11\rangle} = \langle Z \rangle_{|1\rangle} \langle Z \rangle_{|1\rangle} = (-1)(-1) = 1$$

Note: Pauli strings are *read* from right to left. For ZI, the operator I applies to qubit 0 and Z to qubit 1.

QEst Hack 2026

Pauli Strings

An arbitrary Hamiltonian H can be expressed as a sum of *Pauli strings*.

For example: $H = ZZZ + 1.2\,IZZ - 0.6\,ZIZ + 0.3\,ZZI$.

```python
from qiskit.quantum_info import SparsePauliOp

H = SparsePauliOp(
  data=["ZZZ", "IZZ", "ZIZ", "ZZI"],
  coeffs=[1.0, 1.2, -0.6, 0.3]
    )

print(H)
```

```
SparsePauliOp(['ZZZ', 'IZZ', 'ZIZ', 'ZZI'],
              coeffs=[ 1. +0.j,  1.2+0.j, -0.6+0.j,  0.3+0.j])
```

Estimating the expectation value

In practice, the expectation value of an observable is obtained by preparing and measuring the circuit multiple times.

$$\langle H \rangle_\psi = \langle ZZZ \rangle_\psi + 1.2 \langle IZZ \rangle_\psi - 0.6 \langle ZIZ \rangle_\psi + 0.3 \langle ZZI \rangle_\psi.$$

Qiskit offers the Estimator primitive to compute expectation values!

```python
from qiskit_ibm_runtime.executor_estimator import Estimator

qc = QuantumCircuit(3) ; qc.h([0, 1, 2])

estimator = Estimator(simulator)
result = estimator.run([(qc, H)]).result()

print(f"Expectation value {result[0].data.evs:.4f}")
```

```
Expectation value 0.0857
```

Summary

- A quantum measurement returns **only one** of the exponential number of basis states.
- The measurement outcome is **probabilistic**, and the probability associated to one basis state is given by the **square norm of its coefficient**.
- The **Hamiltonian** is the operator associated to the energy of a quantum system, and its **expectation value** is a measure of the “average energy” contained in the system.
- In quantum algorithms, Hamiltonians are commonly expressed as a **linear combinaison of Pauli strings**.

Quantum algorithms

QEst Hack 2026

Grover’s Algorithm

Grover’s algorithm searches an **unstructured database** with a **quadratic advantage** over any classical algorithm.

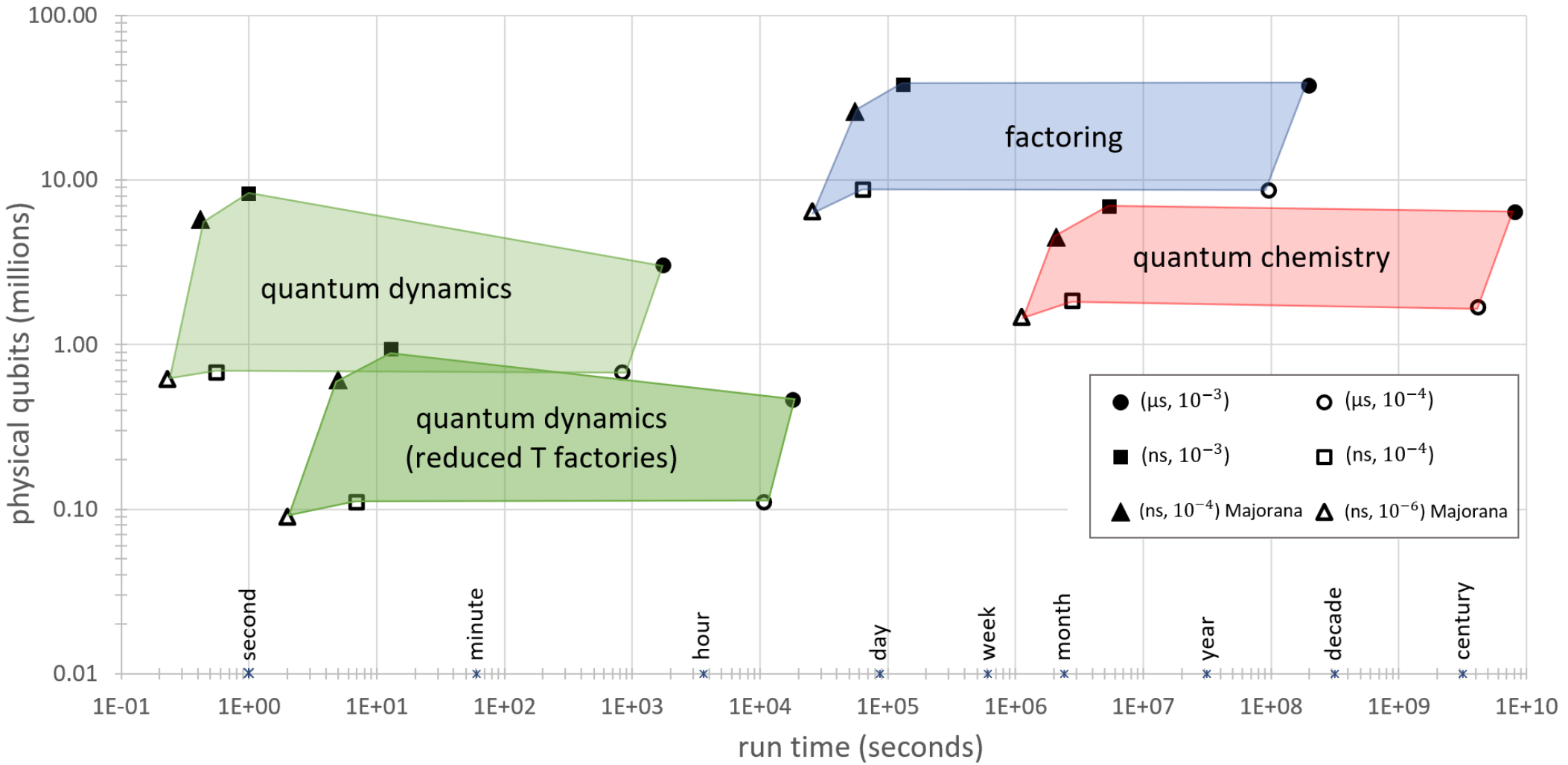
QEst Hack 2026

Shor’s Algorithm

Shor’s algorithm efficiently determines the **periodicity** of a function related to the factorization problem, which forms the basis of protocols **commonly used in cryptography**.

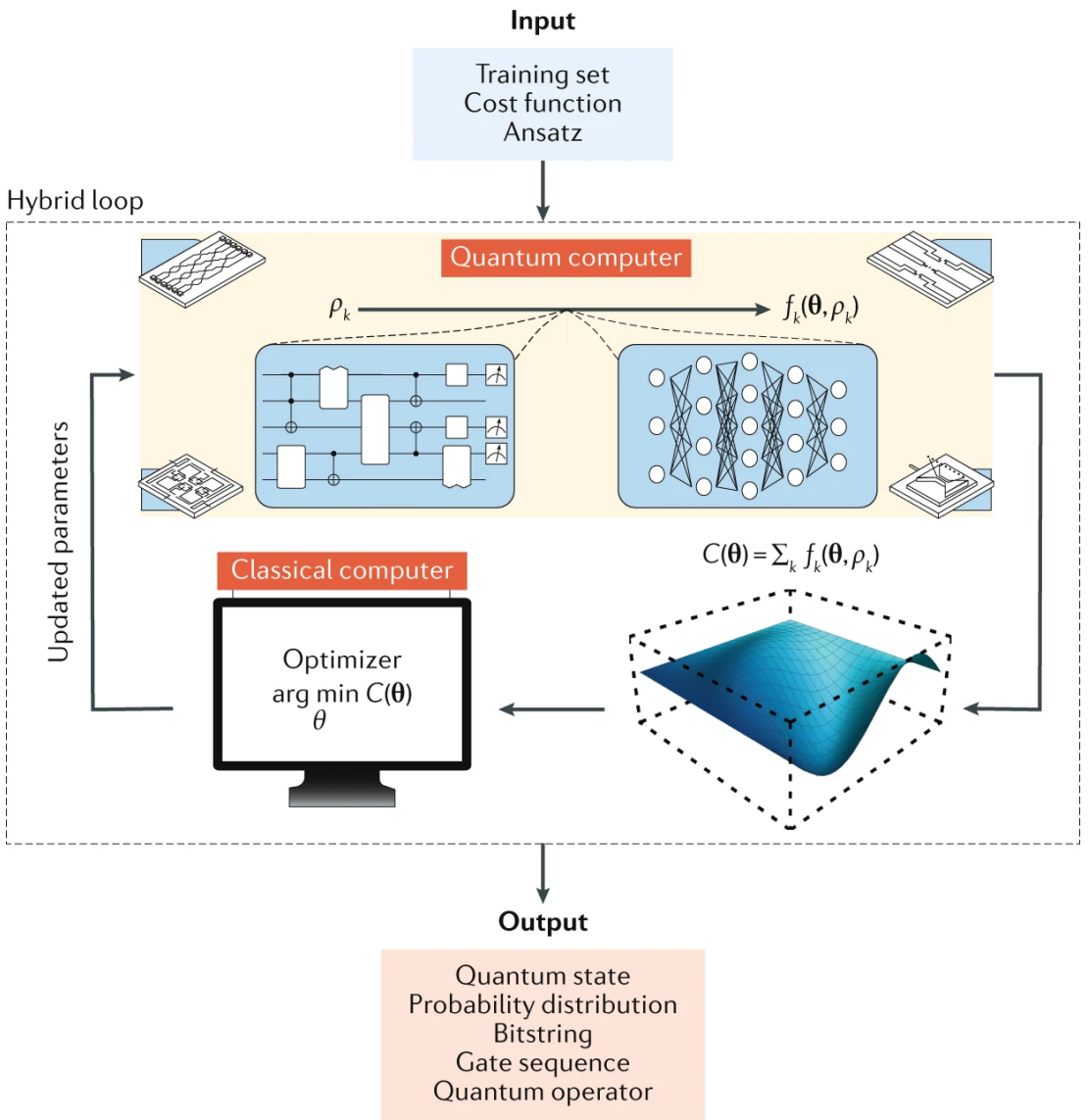
QEst Hack 2026

Resource estimation



Source: arXiv:2211.07629

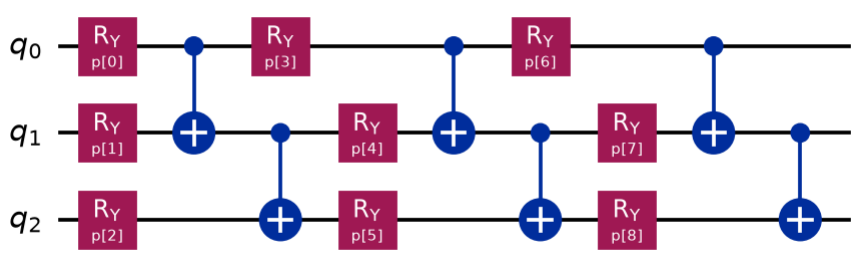
Variational quantum algorithms (VQA)



Source: https://doi.org/10.1038/s42254-021-00348-9

Parameterized circuit (ansatz)

```python
from qiskit.circuit import ParameterVector, QuantumCircuit

nb_layers = 3
nb_qubits = 3
qc = QuantumCircuit(nb_qubits)
params = ParameterVector("p", nb_layers * nb_qubits)

for layer in range(nb_layers):
    for q in range(nb_qubits):
        qc.ry(params[q + layer * nb_qubits], q)
        if q != 0:
            qc.cx(q - 1, q)
qc.draw('mpl', scale=0.7)
```



Cost function

- The **cost function** is a measure of how good a current solution is.
- The *current solution* is entirely decribed by the value taken by the parameters of the problem.
- The goal is to find the parameter values that minimize this function.

$$f(\boldsymbol{\theta}) = \langle \psi(\boldsymbol{\theta}) | H | \psi(\boldsymbol{\theta}) \rangle$$

Optimizer

An algorithm that minimizes a parameterized cost function.

Some methods do not require a gradient, for example:

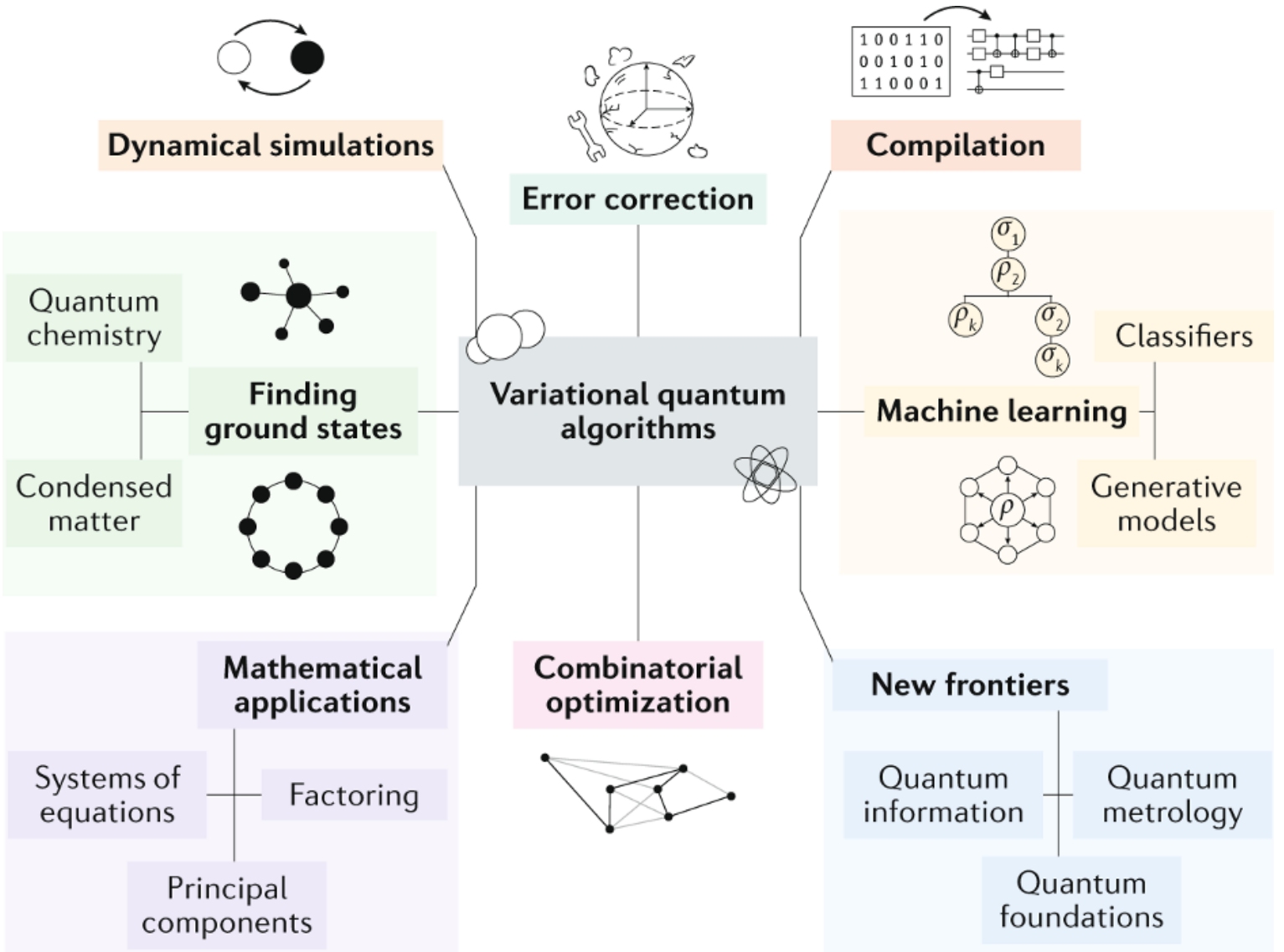
- COBYLA
- SPSA

Other methods require computing the gradient of the parameterized circuit, for example:

- BFGS
- Adam

Calculating the gradient of a parameterized quantum circuit requires $\mathrm{O}(N_{\mathrm{params}})$ **circuit evaluations / iteration**.

Applications



Source: https://doi.org/10.1038/s42254-021-00348-9

Summary

- Quantum algorithms offer advantage over classical algorithms for **some very specific problems**.
- The problems for which there is a known quantum advantage necessitate a **large number of qubits** and error rates **much lower than what current devices offer**.
- In the meantime, VQAs form an interesting playground to **experiment with quantum computing**.
- VQAs explore the space of quantum states through **parametrized quantum circuits**. The parameters are updated iteratively in a **hybrid quantum-classical loop**.

Quantum combinatorial optimization

QEst Hack 2026

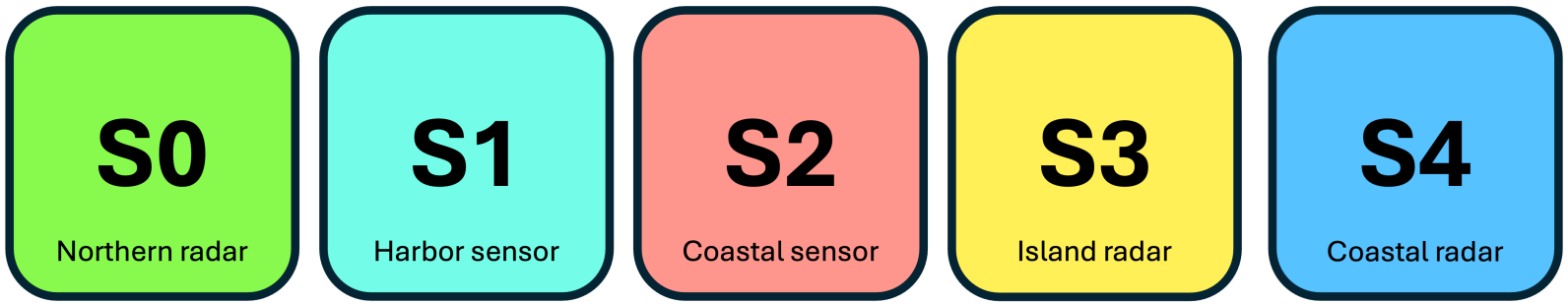
Definition

Wikipedia:

> *A combinatorial optimization problem is to find, within a discrete set, one of the best feasible solutions, where “best” is defined by an objective function.*

> *The set of feasible solutions cannot be exhaustively enumerated; the difficulty lies precisely in the fact that their number makes enumeration impractical.*

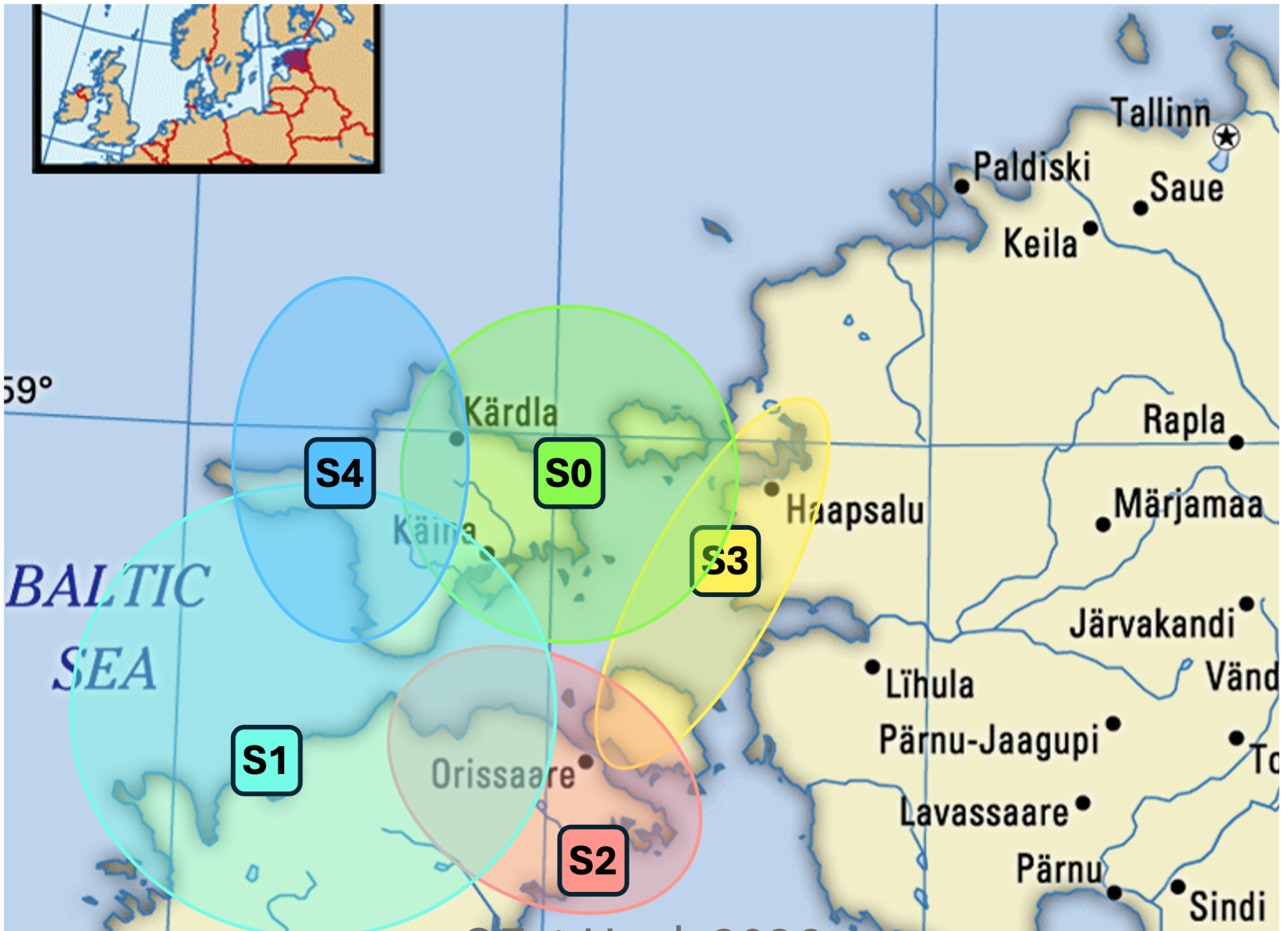
Toy problem description



A coastal surveillance network must schedule the maintenance of its five radar and sensor stations, requiring the stations to be powered down during one of two periods: 9:00 AM or 11:00 AM.

If two stations have overlapping coverage, **they should be assigned to different periods whenever possible in order to minimize surveillance blind spots.**

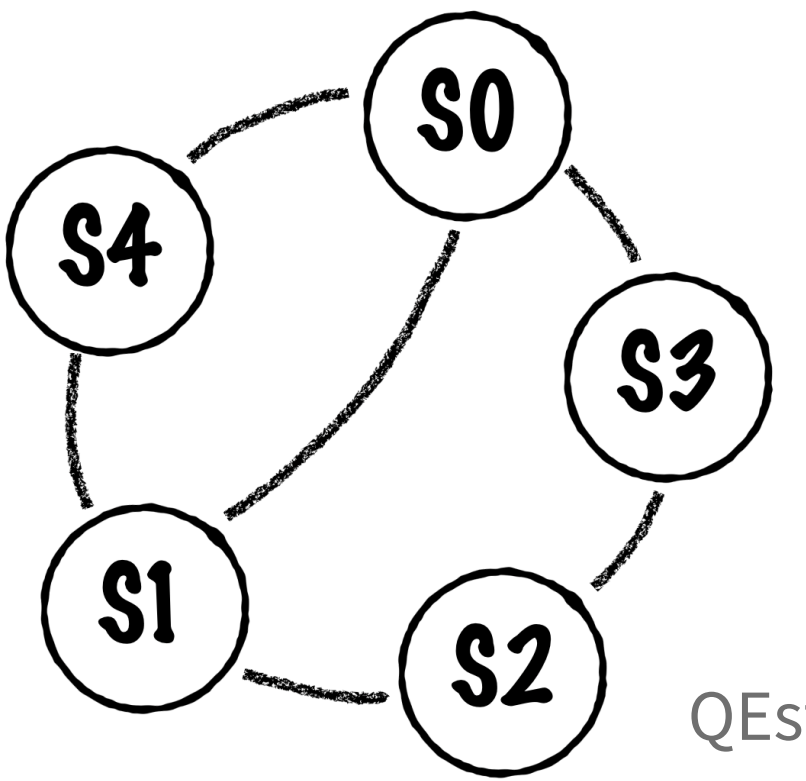
Toy problem map

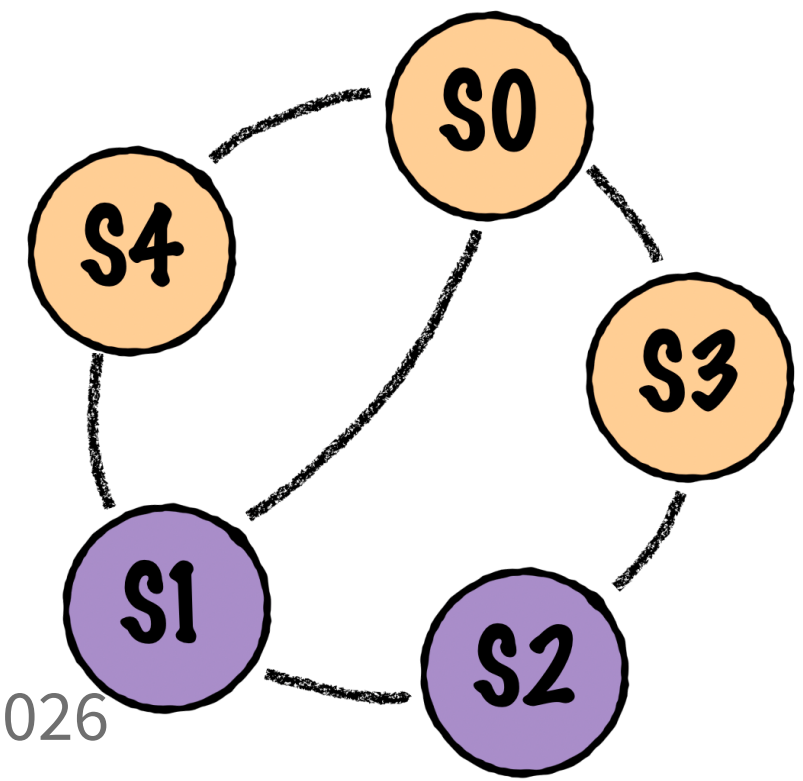


Graph representation

The stations are the vertices of a graph; edges connect stations whose coverage overlap.

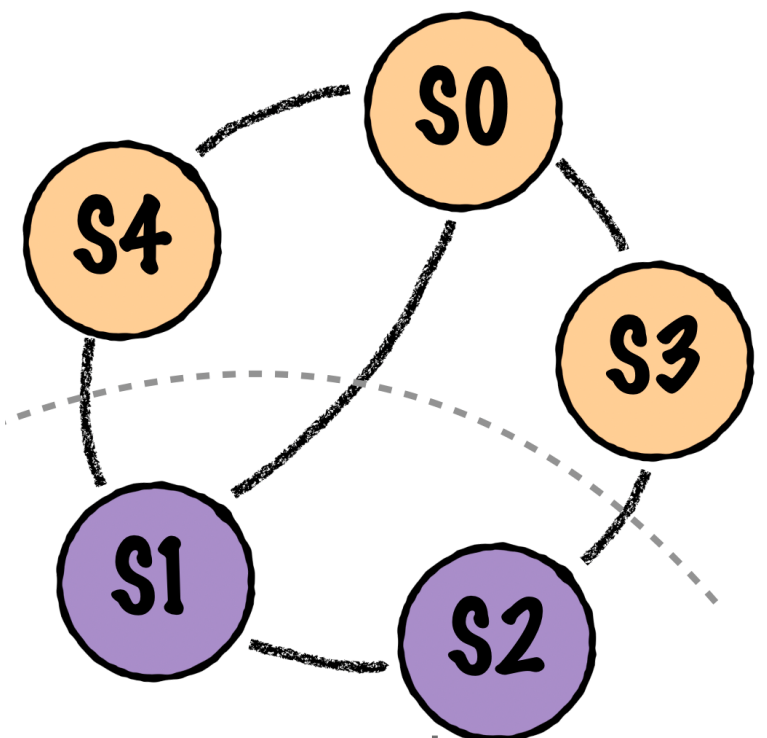
The two maintenance periods, 9:00 AM and 11:00 AM, are labeled orange (0) and indigo (1).





Maximum cut (Max-Cut)

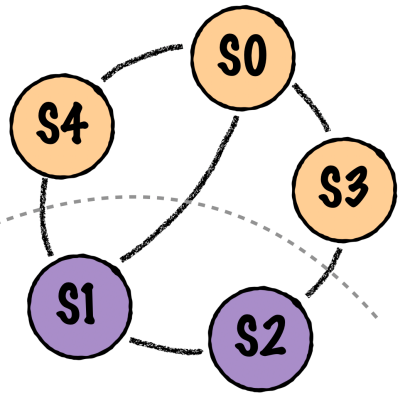
When two stations with coverage overlap are assigned to different periods, the corresponding edge is *cut*. The set of cut edges forms a *cut*. The cut that contains the largest number of edges is the *maximum cut* (*max-cut*).



Encoding

- The maintenance period of each station is encoded with the variable $m_i \in \{0, 1\}$, with 0 = ‘orange’ (9:00) and 1 = ‘indigo’ (11:00).
- The indices $i \in \{0, 1, 2, 3, 4\}$ represent the stations $\{$S0, S1, S2, S3 and S4$\}$, respectively.

We define the vector $\boldsymbol{M} = [m_0, m_1, m_2, m_3, m_4]$.

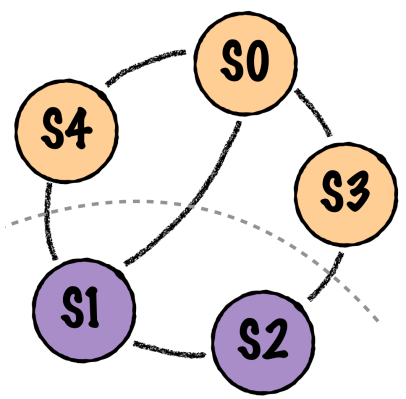


For this example, $M = [0, 1, 1, 0, 0]$.

Cost of a cut

To find the maximum cut, we define a function $g(i, j)$ on edges. For an edge (i, j) between vertices i and j,

$$g(i,j) = \begin{cases} +1, & \text{if } m_i = m_j \\ -1, & \text{if } m_i \neq m_j. \end{cases}$$



For this cut, we have

- $g(0,3) = g(0,4) = g(1,2) = +1$
- $g(0,1) = g(1,4) = g(2,3) = -1$

The maximum cut is obtained by minimizing the function $G = \sum_{(i,j)} g(i,j)$.

Brute-force approach

Exhaustively test all configurations and choose the best one.

B5	B4	B3	B2	B1	G
0	0	0	0	0	6
0	0	0	0	1	0
0	0	0	1	0	0
0	0	0	1	1	-2
0	0	1	0	0	2
0	0	1	0	1	-4
0	0	1	1	0	0
0	0	1	1	1	-2
0	1	0	0	0	2
0	1	0	0	1	0
0	1	0	1	0	-4
0	1	0	1	1	-2
0	1	1	0	0	2
0	1	1	0	1	0
0	1	1	1	0	0
0	1	1	1	1	2

B5	B4	B3	B2	B1	G
1	0	0	0	0	2
1	0	0	0	1	0
1	0	0	1	0	0
1	0	0	1	1	2
1	0	1	0	0	-2
1	0	1	0	1	-4
1	0	1	1	0	0
1	0	1	1	1	2
1	1	0	0	0	-2
1	1	0	0	1	0
1	1	0	1	0	-4
1	1	0	1	1	2
1	1	1	0	0	-2
1	1	1	0	1	0
1	1	1	1	0	0
1	1	1	1	1	6

Brute-force approach

Exhaustively test all configurations and choose the best one.

For a problem with 5 vertices

> $2^5 = 32$ configurations

For a problem with 100 vertices

> $2^{100} \sim 10^{30}$ configurations
>
> $(1, 267, 650, 600, 228, 229, 401, 496, 703, 205, 376)$

B5	B4	B3	B2	B1	G
0	0	0	0	0	6
0	0	0	0	1	0
0	0	0	1	0	0
0	0	0	1	1	-2
0	0	1	0	0	2
0	0	1	0	1	-4
0	0	1	1	0	0
0	0	1	1	1	-2
0	1	0	0	0	2
0	1	0	0	1	0
0	1	0	1	0	-4
0	1	0	1	1	-2
0	1	1	0	0	2
0	1	1	0	1	0
0	1	1	1	0	0
0	1	1	1	1	2

B5	B4	B3	B2	B1	G
1	0	0	0	0	2
1	0	0	0	1	0
1	0	0	1	0	0
1	0	0	1	1	2
1	0	1	0	0	-2
1	0	1	0	1	-4
1	0	1	1	0	0
1	0	1	1	1	2
1	1	0	0	0	-2
1	1	0	0	1	0
1	1	0	1	0	-4
1	1	0	1	1	2
1	1	1	0	0	-2
1	1	1	0	1	0
1	1	1	1	0	0
1	1	1	1	1	6

Quantum cost function

QEst Hack 2026

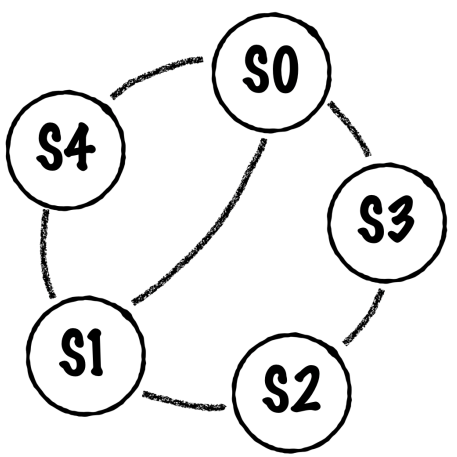
We assign one qubit to each vertex and prepare the state $|\psi\rangle = |q_4 q_3 q_2 q_1 q_0\rangle$, where $q_i \in \{0, 1\}$.

This allows us to implement our cost function:

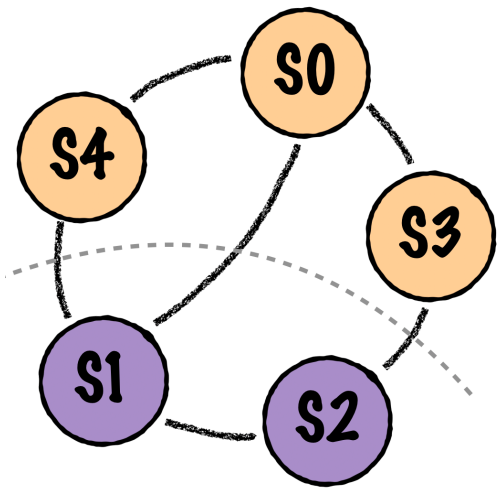
$$G = \sum_{i,j \in L} g(i,j)$$
$$= \langle H \rangle_\psi$$
$$= \sum_{(i,j)} \langle Z_i Z_j \rangle_\psi .$$

```
H = SparsePauliOp(
        data=["IIIZZ",
              "IZIIZ",
              "ZIIIZ",
              "IIZZI",
              "ZIIZI",
              "IZZII"]
               )
print(H)
```

```
SparsePauliOp(['IIIZZ', 'IZIIZ',
'ZIIIZ', 'IIZZI', 'ZIIZI', 'IZZII'],
              coeffs=[1.+0.j, 1.+0.j,
1.+0.j, 1.+0.j, 1.+0.j, 1.+0.j])
```



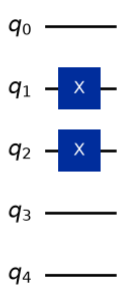
Example 1



$$|\psi\rangle = |00110\rangle$$

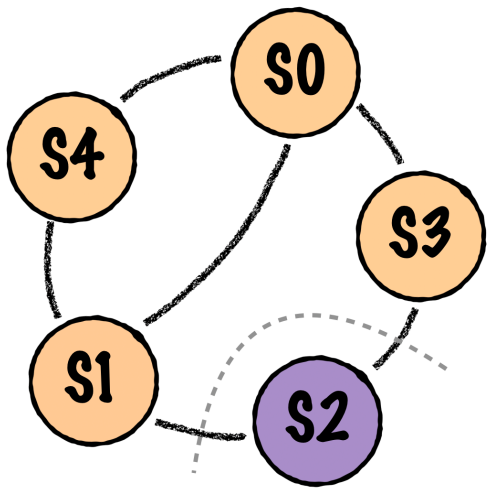
```python
H = SparsePauliOp(
    data=["IIIZZ", "IZIIZ", "ZIIIZ",
      "IIZZI", "ZIIZI", "IZZII"])

qc = QuantumCircuit(5)
qc.x(1)
qc.x(2)

result = estimator.run([(qc, H)]).result()

print(f"Expectation value {result[0].data.e
qc.draw('mpl', scale=0.5)
```

```
Expectation value 0.0000
```



QEst Hack 2026

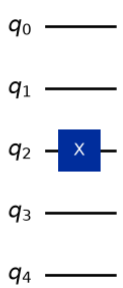
Example 2



$$|\psi\rangle = |00100\rangle$$

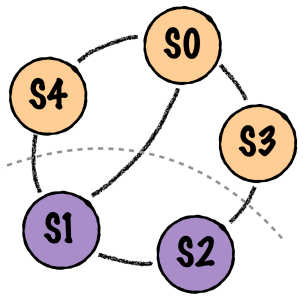
```python
H = SparsePauliOp(
    data=["IIIZZ", "IZIIZ", "ZIIIZ",
      "IIZZI", "ZIIZI", "IZZII"])

qc = QuantumCircuit(5)
qc.x(2)

result = estimator.run([(qc, H)]).result()

print(f"Expectation value {result[0].data.e
qc.draw('mpl', scale=0.5)
```

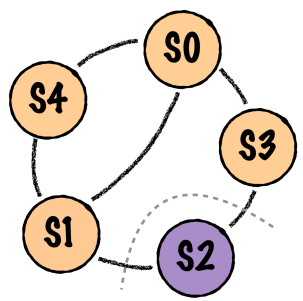
```
Expectation value 2.0000
```



QEst Hack 2026

Example 3

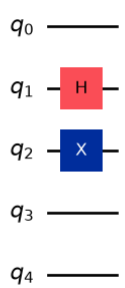





```python
H = SparsePauliOp(
    data=["IIIZZ", "IZIIZ", "ZIIIZ",
      "IIZZI", "ZIIZI", "IZZII"])

qc = QuantumCircuit(5)
qc.h(1)
qc.x(2)

result = estimator.run([(qc, H)]).result()

print(f"Expectation value {result[0].data.e
qc.draw('mpl', scale=0.5)
```

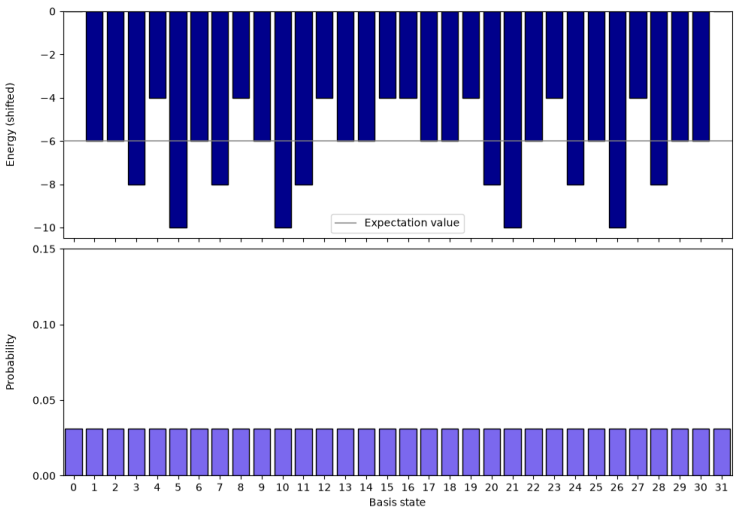
```
Expectation value 1.0176
```

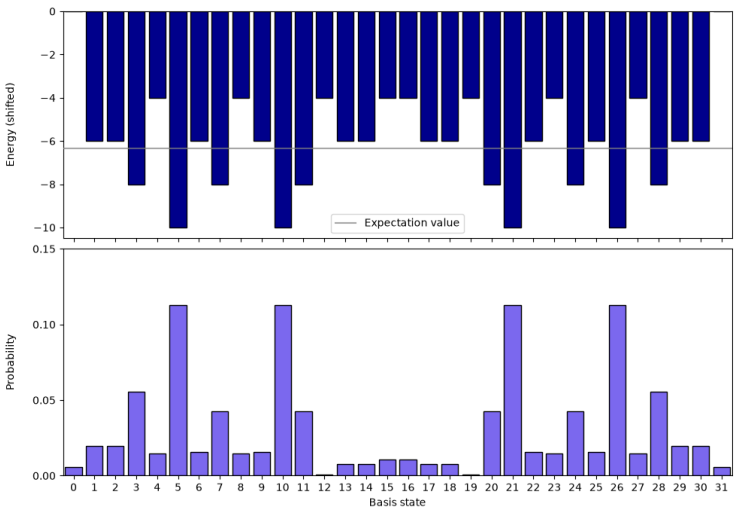
$$|\psi\rangle = \frac{1}{\sqrt{2}}(|00110\rangle + |00100\rangle)$$

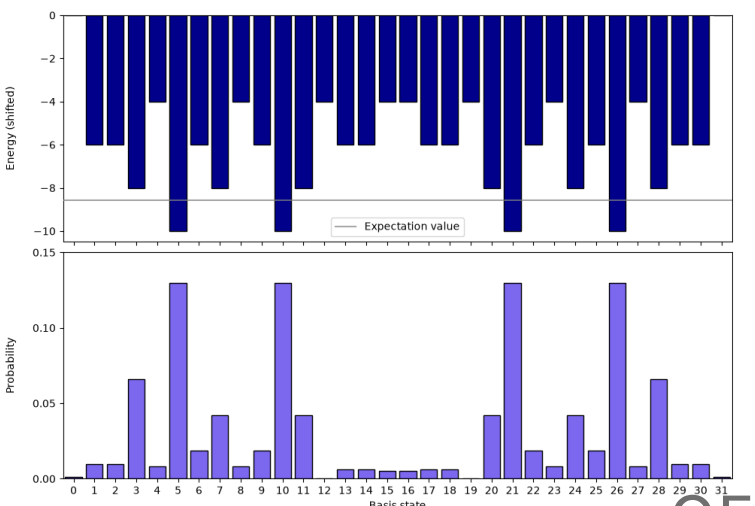


QEst Hack 2026

Average cost and statevector







Objective:

Find the circuit preparing the quantum state that minimizes the expectation value.

QAOA

Quantum Approximate Optimization Algorithm (QAOA)

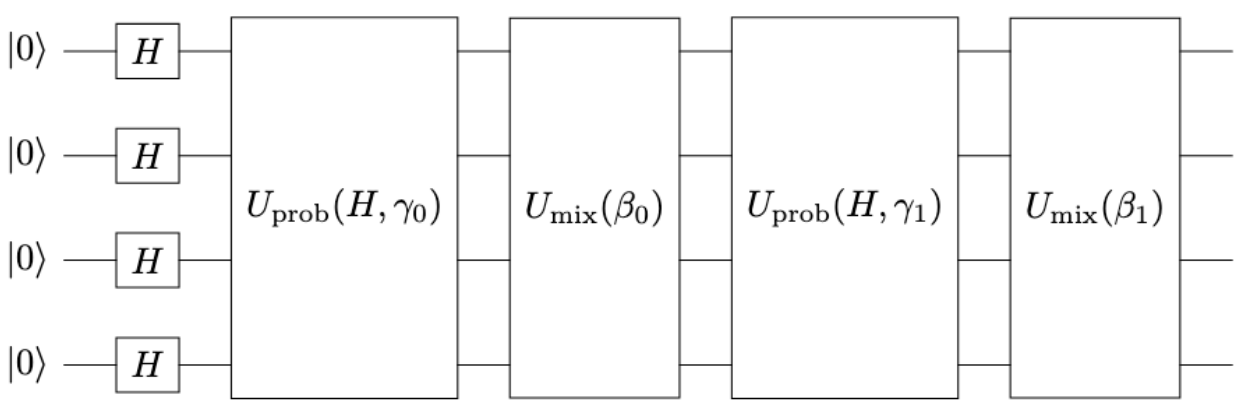
- QAOA is an algorithm for finding the state that minimizes the cost function G.
- QAOA uses a quantum circuit containing
  - parameters to be optimized
  - an encoding of the problem's Hamiltonian.
- By preparing a state that minimizes G, we maximize the probability of measuring a "good solution".

Circuit structure

A QAOA circuit consists of two sub-circuits:

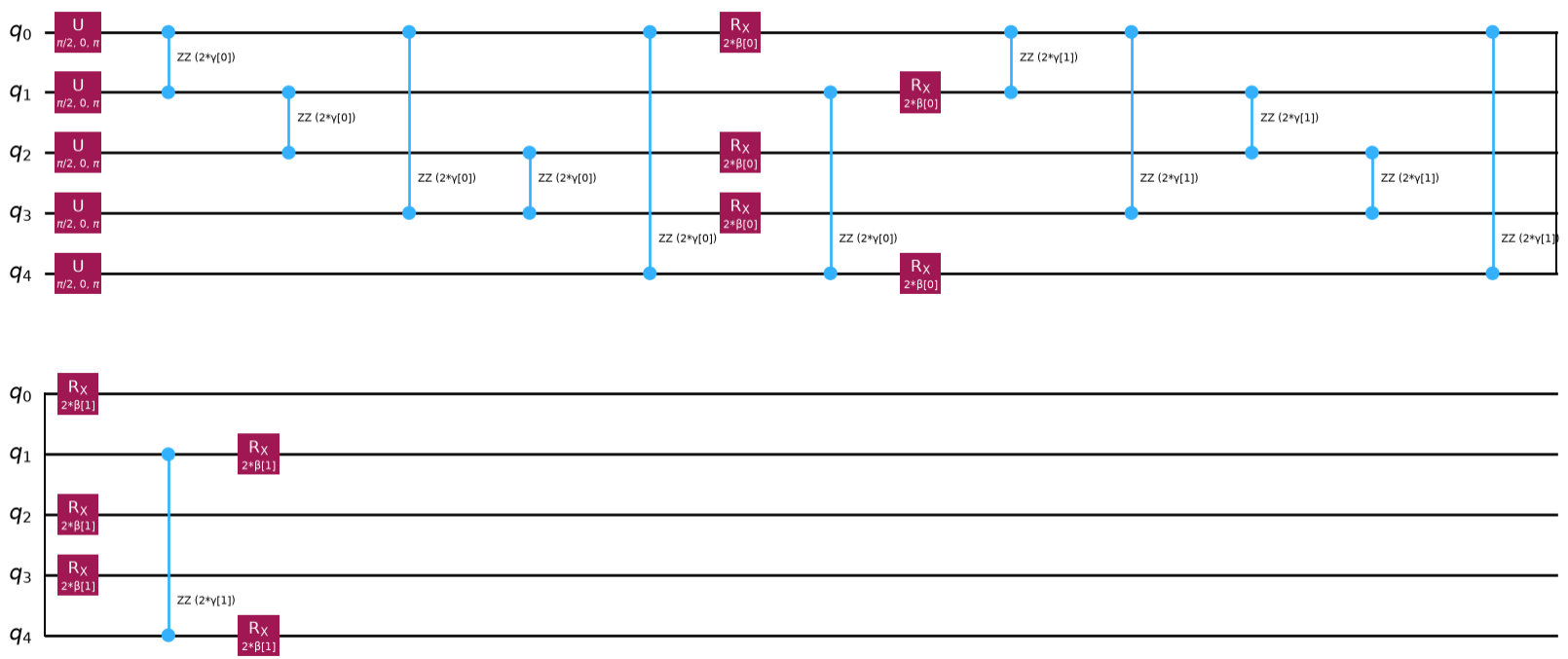
1. a *mixer* circuit, $U_{\text{mix}}(\beta)$
2. a *problem* circuit, $U(H_{\text{prob}}, \gamma)$

These two sub-circuits are alternated p times. For example, with $p = 2$:



Qiskit QAOA circuit

```
1 from qiskit.circuit.library import QAOAAnsatz
2
3 p = 2
4 circuit_qaoa = QAOAAnsatz(H, reps=p)
5
6 circuit_qaoa.decompose(reps=2).draw('mpl', scale=0.5)
```



QEst Hack 2026

Optimizing a QAOA circuit

```python
from scipy.optimize import minimize

def cost_function(params, estimator, circuit, H):
    """
      Args:
        params (np.ndarray): Parameters of the variational circuit.
        estimator (EstimatorV2): Qiskit primitive to compute expectation va
        circuit (QuantumCircuit): Parametrized quantum circuit.
        H (SparsePauliOp): Hamiltonian of the problem.
    """

    result = estimator.run([(circuit, H, params)]).result()
    cost = result[0].data.evs
    return cost

initial_params = np.zeros(circuit_qaoa.num_parameters)
opt_res = minimize(cost_function,
                   initial_params,
```

```
Optimal cost found: -1.037109375
```

Objectives of the QEst-Hack

- Learn about the basics of quantum computing
- Discuss with mentors about the state of quantum computing
- Take the first steps in mapping a problem on a quantum computer
- Assess how quantum computing could impact your field of activity
- Have fun 😀

Grading for Challenge 1

60% of total grade.

Evaluation Criteria	Points
Explanation of the solution by the representative	12
Expectation value with optimal parameters	8
Overlap of the optimal state with optimal solution	8
Clarity of the explanations in the notebooks	8
Run on real hardware	5
Lessons learned	4
Total	**45**

Your mentors!

Source code

https://github.com/algolab-quantique/qest-hack26

QEst Hack 2026